

Yini

Documentación del proyecto

Asistente fundamentado sobre documentos de seguros para revisión del asesor

Versión de documentación generada desde el repositorio

Fuente: README y documentación operativa del proyecto en /docs

Fecha: 2026-06-22

1. Resumen ejecutivo

Yini es un asistente fundamentado para asesores de seguros. Su objetivo es responder preguntas operativas y comerciales a partir de documentos oficiales del corpus, manteniendo la respuesta vinculada a evidencia recuperada desde el RAG. El sistema no sustituye la revisión del asesor: genera borradores citados para acelerar consulta, revisión y orientación interna.

El MVP actual está desplegado sobre una interfaz Gradio en Hugging Face Spaces, usa Groq para generación de respuesta, Qdrant Cloud para recuperación vectorial y embeddings multilingües para consulta semántica en español.

2. Problema que resuelve

El equipo asesor necesita consultar múltiples documentos de seguros, procesos, políticas, preguntas frecuentes e instructivos. Esa consulta suele implicar:

- localizar rápidamente el documento correcto dentro de un corpus grande;
- encontrar el fragmento exacto relevante para una pregunta;
- sintetizar una respuesta utilizable sin perder trazabilidad;
- diferenciar entre cobertura, proceso, política y restricción;
- mantener un criterio de revisión humana antes de usar la información.

Yini reduce ese tiempo de búsqueda y estructuración al producir una respuesta sugerida basada en citas del corpus.

3. Objetivo del MVP

El MVP tiene un objetivo acotado:

- responder preguntas en español sobre categorías documentales ya onboardadas;
- mostrar una respuesta sugerida con evidencia citada;
- operar sobre corpus PDF ya procesado;
- permitir una validación funcional hosted en Hugging Face Spaces;
- mantener una compuerta determinística de release para evitar regresiones del corpus y del backend.

No es objetivo del MVP:

- soportar formularios .docx;
- automatizar onboarding de nuevas categorías sin validación operativa;

- reemplazar al asesor en decisiones comerciales o normativas;
- ejecutar una experiencia multiusuario compleja o un panel de administración.

4. Alcance funcional vigente

La línea base de go-live actual cubre estas familias documentales:

- ARL
- MOVILIDAD / AUTOS
- MOVILIDAD / BICICLETAS Y PATINETAS
- MOVILIDAD / MOTOS
- MOVILIDAD / TRANSVERSALES / choque simple
- MOVILIDAD / PV
- MOVILIDAD / UTILITARIO Y PESADOS
- MOVILIDAD / FINANCIACION
- MOVILIDAD / VIAJES
- MOVILIDAD / SUSCRIPCION
- MOVILIDAD / MUEVETE LIBRE
- MOVILIDAD / SOAT
- EPS / PLAN COMPLEMENTARIO PAC

Estas categorías son las que actualmente deben considerarse disponibles para pruebas del MVP.

5. Arquitectura de alto nivel

La solución sigue un flujo RAG clásico con decisiones operativas específicas para este proyecto:

1. **Corpus raw:** los documentos PDF se almacenan en `data/raw/` preservando la taxonomía por carpeta.
2. **Ingesta:** el pipeline convierte PDFs a markdown limpio.
3. **Chunking:** el markdown se divide en fragmentos con metadata documental.
4. **Embeddings:** cada chunk se vectoriza con un modelo multilingüe.
5. **Indexación:** los embeddings y su metadata se cargan en Qdrant Cloud.
6. **Retrieval:** una consulta del asesor activa búsqueda vectorial con filtros y equivalencias curadas.
7. **Generación fundamentada:** Groq compone la respuesta final a partir de evidencia recuperada.

8. **Interfaz:** la UI Gradio presenta la respuesta sugerida, el estado de revisión y detalles expandibles.

6. Componentes principales

6.1 Interfaz

- **Superficie:** Gradio.
- **Objetivo:** exponer una experiencia simple para consulta del asesor.
- **Salida principal:** respuesta sugerida con citas y estado de revisión.

6.2 Backend de respuesta

- **Proveedor LLM:** Groq.
- **Modelo configurado actualmente:** openai/gpt-oss-120b.
- **Función:** redactar una respuesta final utilizando únicamente la evidencia recuperada.

6.3 Capa de embeddings

- **Proveedor:** sentence-transformers.
- **Modelo configurado actualmente:**
sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2.
- **Motivo:** alineación con consultas y corpus en español.

6.4 Base vectorial

- **Proveedor:** Qdrant Cloud.
- **Colección actual:** yini-policies.
- **Uso:** recuperación vectorial y filtrado por metadata documental.

7. Flujo operativo del RAG

El flujo operativo completo para una categoría es:

1. ubicar los PDFs nuevos dentro de data/raw/ sin mezclar categorías;
2. ejecutar warmup de runtime de batch cuando aplique;
3. correr la ingesta de la categoría objetivo;
4. inspeccionar el markdown generado;
5. inspeccionar la calidad de chunks y metadata;

6. generar embeddings solo de la categoría afectada;
7. indexar esos embeddings en Qdrant;
8. validar retrieval real con preguntas de negocio o proceso;
9. validar una respuesta fundamentada real;
10. abrir un slice correctivo si la categoría queda ingestada pero no usable.

En este proyecto, “ingestado” no equivale a “listo para producción”.

8. Regla de estructuración del corpus

Los documentos raw deben conservar su estructura por categoría. La práctica correcta es mantener carpetas semánticas dentro de `data/raw/` y evitar aplanar los archivos en un único directorio.

Razones:

- la taxonomía queda reflejada en `source_pdf_id`;
- el filtrado por categoría es más estable;
- el debugging de retrieval resulta más trazable;
- las remediaciones pueden acotarse por cohort o categoría.

9. Restricciones del corpus

El MVP actual mantiene estas reglas:

- solo se onboardean PDFs;
- los formularios Word .docx están fuera de alcance;
- los documentos pesados o con diagramación compleja pueden requerir cohorts aislados;
- las tablas, comparativos y diagramaciones deben validarse después de la ingesta, no asumirse correctas por defecto.

10. Variables y configuración de entorno

Para ejecución local y hosted, la línea base requiere al menos:

- `GROQ_API_KEY`
- `GROQ_MODEL`
- `QDRANT_URL`
- `QDRANT_API_KEY`

El repositorio incluye `.env.example` como referencia de configuración reproducible sin exponer credenciales reales.

11. Reproducción local

La reproducción local sigue esta lógica:

- usar `.venv` como entorno principal de desarrollo;
- instalar dependencias desde `pyproject.toml` o `requirements.txt`;
- usar Docker como baseline de reproducibilidad y despliegue, no como loop primario de desarrollo;
- ejecutar la UI con `python -m app.ui`;
- usar `make` y la CLI de `rag.ingestion` para las tareas operativas de batch.

12. Despliegue hosted

El deployment vigente del MVP usa Hugging Face Spaces con SDK Docker.

La lógica general es:

1. construir imagen desde `Dockerfile`;
2. instalar el paquete del proyecto;
3. exponer la aplicación Gradio;
4. inyectar secretos por variables del Space;
5. validar smoke hosted posterior al deploy.

13. Validación del MVP

La compuerta formal previa a release es determinística y local:

- comando base: `make test-release`

Esta compuerta protege la línea base aceptada del MVP y evita depender de validaciones live frescas cada vez que se necesita decidir si el estado es liberable.

Además de esa compuerta, el proyecto usa:

- smoke acceptance por categorías vigentes;
- validación puntual live contra Qdrant y Groq después de cambios relevantes;
- validación operativa por cohort cuando se onboardea una nueva categoría.

14. Categorías disponibles para compartir en el MVP

El MVP puede compartirse actualmente para pruebas focalizadas sobre:

- ARL
- Movilidad: autos, motos, bicicletas y patinetas, SOAT, viajes, financiación, suscripción, propuesta de valor, utilitarios y pesados, Muévete Libre y transversales de choque simple
- EPS / Plan Complementario PAC

La recomendación es no anunciar capacidades sobre categorías no incluidas expresamente en esta lista.

15. Limitaciones conocidas

Las principales limitaciones del estado actual son:

- el sistema responde solo sobre el corpus onboardado y validado;
- la calidad depende de la extracción documental y del chunking;
- ciertos documentos con tablas o diagramación densa pueden requerir remediaciones específicas;
- la respuesta sigue siendo un borrador para revisión del asesor;
- los .docx no forman parte del corpus ni de la superficie de respuesta del MVP.

16. Proceso recomendado para nuevas categorías

La experiencia del proyecto consolidó una política operativa:

- incorporar una categoría a la vez;
- hacer onboarding por cohorts pequeños cuando exista riesgo documental;
- validar markdown, chunks, retrieval y answer antes de promover la categoría;
- usar equivalencias de términos y overlays de metadata cuando la recuperación no refleje bien el lenguaje del negocio;
- abrir slices correctivos estrechos cuando el problema es de extracción, ranking o metadata.

17. Estado actual del proyecto

El proyecto ya cuenta con:

- backend funcional para retrieval y respuesta fundamentada;
- corpus MVP onboardado para las categorías listadas;

- interfaz Gradio en español y lista para validación con usuarios;
- baseline documental en README y docs/;
- baseline de reproducibilidad local y hosted.

18. Próximos pasos razonables

Los siguientes trabajos recomendables, después del MVP, son:

- ampliar cobertura documental por nuevas categorías priorizadas;
- endurecer todavía más la validación visual de respuestas tabulares;
- seguir cerrando gaps de acoplamiento técnico cuando no comprometan el avance del corpus;
- evaluar soporte futuro para nuevas superficies o tipos documentales solo después de estabilizar completamente la operación del corpus PDF.

19. Conclusión

Yini ya dispone de una base MVP consistente para consulta asistida sobre documentos de seguros en español. El valor actual no está en responder “todo”, sino en responder bien sobre un corpus delimitado, con evidencia citada, proceso de onboarding trazable y una ruta clara de validación antes de promover nuevas categorías.